Question	Answer
Can we say that whenever we use past data to create a model (because we knew the X and Y both), it will always be supervised learning?	Yep, when Y is known from past it is supervised
what is the diff between feature space and level space	Feature space (X): the space of inputs; all the measurable attributes describing your data points (e.g., height, weight, pixel values). Each data point is a vector living in this space. Label space (Y): the space of outputs; what you're trying to predict (e.g., class {0,1}, a price, a category).
Pls give a real life example for unspervised learning . Would we not always go with supervised ?	when you want to categorize customers based on their movie watching behavior (NETFLIX)
Can we say regression models are always made from supervised learning?	Yes linear, logistic, poisson etc are supervised since Y is known
can you use some real project examples?	e.g Predicting the price of the uber ride given the rain conditions, distance, supply, demand etc
So the model is nothing but a mathematical quation? Is it possible to see that equation?	Yeah, pretty much. like y = w0 + w1x1 + w2x2 (linear regression) or y = sign (W.X) (linear classifier/SVM).
what is the diff between feature space and level space	label space you mean? If so here is the difference : Feature space is the math space formed by the input variables (if there are 5 inputs then 5D space) of a dataset, whereas label space is the space representing the target outputs, categories, or labels that a model tries to predict
By feature vectors, we mean the predictors and their co-efficient ?	feature vector = the data point, weight vector = what the model learns.
This Well explained...can someone please tell how a long sentence as input is calculated in terms on function....is each word we feed is considered as X value....	words aren't numbers by default, so there's a conversion step first: each word gets turned into a number/vector through some encoding (one-hot, bag-of-words, TF-IDF, or more modern "embeddings" like Word2Vec/GloVe where each word fed as a dense vector of numbers). each word's vector essentially becomes part of the X. how they're combined into "X" depends heavily on the model architecture
Pls give a real life example for unspervised learning . Would we not always go with supervised ? Renuka S_TA 10:10 AM when you want to categorize customers based on their movie watching behavior for eg in NEtflix >> Referring this ? How can Netflix classification be unsupervise . cause we have past data and we can label that ?	if the Ys are not previously defined then it is unsupervised. Netflix has too many dimensions like time of viewing, characteristics of the person viewing, the kind of movies watched entirely, the kind of movies not watched for >5 min etc. When a new user enters, just from the few clicks they are able to recommend an accurate movie, because they mapped the new user's clicks to one of the unsupervised categories they already mapped using K means or one such algorithm. So the label is not there, the behavior (some 100+ variables)are mapped with pre-done unsupervised learning categories and then accurate movie reco is possible

when we are using a higher degree polynomial to predict something . What we mean is : an increase of features that would be used to predict right ? Example for a cat : We look at features like : Eyes , whiskers , fur type , ears this will represented as a : y = w0 + w1x1 + w2x2 .... etc Just want to confirm my understanding	this example is actually describing more features, not a higher degree polynomial. More features (what your cat example shows): adding new distinct measurements Higher degree polynomial: taking the same feature(s) and raising them to powers, or multiplying them together.
Not happy with your answer.	please be specific on which answer you are unhappy with we are here to help you
regarding cat and dog, for some feature, there might be common vector values. then how it can be differentiated?	Agreed there might be some overlap lets say that Puppy and Kitten of same color, so the part of the representation expressing color would be same but part of the representation expressing face shape will be different. And depending on the dataset if we have many such samples where this is happening more weight would be given to part of the representation expressing face
Hi. Again I come today to convert complex definition into simpler statement. Can I say that Supervised = Unsupervised learning + guardrails + validation / Loss?	Nope. Supervised = learning the relationship between The inputs to accurately predict / calculate the output. think of it this way - use as many inputs to predict weather conditions. supervised = function of Xs that minimize the errors in prediction
we are going to go in feature extraction and model training in detail right ?	Yes we will go through all of this in detail. Once we move forward with the course. We will be using Deep Neural Networks as feature extractors
When we try to fit any machine learning model - is the data imputating or preparation is as an important factor for the accuracy	Data preprocessing directly shapes what the model can even learn. Imputation: how you fill missing values (mean/median, model-based, or just dropping rows) can bias your features and quietly hurt accuracy if done carelessly.
so, supervised or unsupervised, they still use these building blocks such as linear/logistic regression/classifiers in their pipeline? for example CNN would be using these building blocks but they will be recurrently applied to decide on the eyes/ears/legs/etc of dog or animal.	supervised models' building blocks are linear (for numerical prediction)/ logistic regression (for classification prediction) . CNN is different algorithm altogether. You need to use a lot of matrix multiplication and pooling concepts to extract features from images and then use Neural network to predict
I assume logistic regression is important when we have the clarify about the pattern - for 1 mathematical expression and fit the entire range of training data	Logistic regression is important because it gives you a probability (0 to 1), not just a hard class label like sign() does; maps any input across the entire range into a smooth curve between 0 and 1.
so this means all classes are divided in different sections ....in a sentence there could be more classes created	Yes thats what we mean when we say modelling decision boundary, we separate them in space.

the pattern can be linear and polinomial and the same pattern is converted to ANN with multiple variants - is that the right understanding?	sort of yes. But Linear models and polynomial models are both "hand-designed" patterns; you decide the form (degree, which terms to include) in advance. A Neural Network (ANN) generalizes this idea: instead of you manually choosing polynomial terms, the network learns complex, non-linear patterns automatically through layers of weights + non-linear activation functions. So it's less "the same pattern converted" and more "ANN is a much more flexible pattern-learner that doesn't need you to pre-specify linear vs. polynomial
In linear regression it create 2 classes , then how it is differnt from classification ? In terms of 2D	linear regression itself does not create classes. Regression predicts how much, classification predicts which class
How does linear classifiers differs from classification or regression?	linear classifier is one of the method used for classification. In classification we predict categories; while in regression we predict numbers
Is complex non-linear distribution accurate?	depends on the use case. When the data is non linear, we can evaluate for the non linearity and then use such models accurately. Using them blindly is not accurate
how to determine - the function to convert the features - so that its separable either linear and nonlinear - is the LDA also used?	there's no single universal formula; it depends on the technique. for non-linear separability, kernels (SVM) or neural nets are the common approaches; LDA is mainly a linear, supervised dimensionality-reduction tool, though a kernelized version exists for non-linear cases too.
Can you provide an example of Overlapping classes ?	Yes, this world is full of people with shades of grey nobody is fully white or fully black (not entirely good or entirely bad), likewise when we do the prediction, there are chances that some observations would belong to more than 1 segment. Like some of price bargainers, can become premium buyers during sale or during some other circumstances. So the segments are defined with many constraints in place
What is overfitting and underfitting?	Overfitting is when in the training data, you try to get the model to high accuracy using non linear methods (When the data might actually be linear(, but when it is tested on new data the accuracy drops. Underfitting is in the training model itself we are assuming linear relationship when the actual relationship is unlinear and the accuracy is low in training itself.
when we encode the best approach to use language specific encoding model - like bert which can also show the semantic similarity in the treebank	yes, BERT (or similar transformer-based embeddings) is the standard best approach today, better than older methods like Word2Vec or GloVe, since it adapts to context rather than giving every word a fixed vector.
what does W.X! mean ? how does factorial comes here	I thing that was W.XT, T for transpose

can you pl explain when feature space is D dimension, W an X are D+1 dimension. I think prof skipped it.	yes, here you go : This comes from how the bias/intercept term gets folded into the weight vector for cleaner math. If your actual features are D-dimensional, say [x1, x2, ..., xD], a linear model's prediction normally looks like w1x1 + w2x2 + ... + wD*xD + b, where b is a separate bias/intercept term. To avoid treating bias separately, people often append a constant 1 to the feature vector X, making it D+1 dimensional: [1, x1, x2, ..., xD], and absorb the bias into W too, making it [b, w1, w2, ..., wD], also D+1 dimensional. This way, the whole equation simplifies into one clean dot product, W·X, instead of writing w·x + b separately, the extra dimension is just a mathematical trick to fold the bias term in, not an actual extra real-world feature.
what is the computational complexity of svm and how many feature variables it can incorporate with practical example	SVM's computational complexity is roughly $O(n^2)$ to $O(n^3)$ depending on kernel type, where n is the number of training samples, this is why SVM struggles to scale well on very large datasets (millions of rows), it gets slow and memory-heavy fast. Feature-wise though, it actually handles high-dimensional data quite well, even thousands of features, since the kernel trick lets it work efficiently in many dimensions without explicitly computing every dimension. Say insta wants to classify if a post is likely to go "viral" or "not viral." Features could include things like number of #, posting time, follower count, caption length, image brightness/color variance, engagement in first hour, and more, easily 50-100+ features. SVM could handle that feature count comfortably since high dimensionality isn't the bottleneck. The real constraint hits on the sample side, if Instagram tried training SVM directly on, say, 10 million posts, then the complexity makes it painfully slow
what are the different type of boundaries in svm and how it is linked with the hyperparameters - is there any mapping?	boundary types are driven by kernel choice: Linear boundary requires linear kernel (straight line/hyperplane, like the basic SVM in the slides) Polynomial boundary requires polynomial kernel (curved boundary, degree controls how curvy) Highly non-linear/blob-like boundary requires RBF/Gaussian kernel (very flexible, wiggly boundaries) so, kernel decides the family of boundary shapes, while C and gamma/degree control how tightly that boundary conforms to the training data within that family. Both together determine overfitting vs. underfitting behavior.

Will the model be pre-labelled in Supervised learning. Then who performs the labelling	Think of it this way, the label could be auto populated or manually labeled, today in most cases it is auto populated. Like did they click an insta ad or not, did they book the uber ride or not. How much did the user end up spending after browsing. What is the temperature after rainfall. What is the stock price etc.
What is this Sign Function? Please Explain. also What are inputs and output of this function.	Wx+ b if the value is -5sign(wx+b) will give -1 if the value is +4 sign(wx+b) = 1, wx+b value we are calculating
What is this Sign Function? Please Explain. also What are inputs and output of this function.	sign(Wx+b) = +1 if Wx+b = positive number sign(Wx+b) = -1 if Wx+b = negative number sign(Wx+b) = 0 if Wx+b = 0
how svm is good for image classification with huge no of features - do we need to perform pca on the image prior to actual business case	SVM handles high-dimensional data well because of the margin maximization idea. PCA reduces noise/redundancy (neighboring pixels are highly correlated), speeds up training significantly, helps avoid overfitting, especially with non-linear kernels (RBF) if you're using a linear kernel with proper regularization (C tuned well), SVM can still perform reasonably even with huge feature counts.
Here, we have only two labels, dogs and cats. What happens when we have more than two labels, such as dogs, cats, and rats? How does the algorithm separate the different classes using decision boundaries and clusters of data points?	With more than two classes, the algorithm typically breaks the problem into multiple simpler comparisons rather than one single boundary line. The two common strategies are One-vs-Rest, where it trains separate boundaries for dog-vs-others, cat-vs-others, and rat-vs-others, then picks whichever boundary gives the highest confidence, and One-vs-One, where it trains a boundary for every pair, like dog-vs-cat, cat-vs-rat, dog-vs-rat, and combines the votes.
what is the real benefit of classifying the data in different classes. and also how to check if our Model is really good or bad...in real life we do not see this output for our inputs	split data into train/test/validation. test data model never sees. compute performance metrics such as accuracy, precision/recall, F1-score, confusion matrix over this held out test data
we will have n different entities, how the model identiifes the feature as they will be completely different. For Shopping, it will be categories that we purchase like Grocery, fashion, mobile Similarly stock market will have its own features. How the model learns these features, do we define that for them	Yes, exactly, features are domain-specific and usually have to be defined by humans upfront, the model doesn't automatically know "categories" or "stock indicators" exist unless you explicitly engineer and feed those as input columns. For shopping bsomeone decides relevant features like purchase category, order frequency, average spend, time since last purchase, these come from business/domain knowledge how will nmodel automatically know that right? For stock market different features entirely, like moving averages, volatility, trading volume, these get defined by financial domain expertise, they both might use similar algorithms underneath fpr all we know
How do we visualize polynomial kernel in n dimensional space - so its not the degree of the single variable x - its the combination of different variable	you can only really "visualize" it for toy cases like 2D→3D. Beyond that, you don't visualize it

so linear classifiers can only be used where there are just 2 variations in dataset?	linear classifiers aren't restricted to 2-class problems overall. This is where the basic version is limited to 2 classes only, since sign() only gives two outputs. for more than 2 classes, there are several other categores such as: One-vs-Rest (OvR): train one linear classifier per class (class A vs. everyone else), pick whichever gives the highest score One-vs-One (OvO): train a classifier for every pair of classes, then vote
update is manual for W	No nothing is manual, the weights are updated automatically by optimization algorithm such as gradient descent
So, if we have 10,000 labels/classes, do we need to create 10,000+ graphs or decision boundaries to separate all the classes? Am I understanding this correctly?	Technically with One-vs-Rest, yes. Since you need some way to distinguish each class from the rest, but practically it's one unified model with multiple outputs, not thousands of separately trained decision boundaries.
Can you please tell some litmus tests to find difference between unsupervised and reinforcement learning with examples? Stocks price prediction would come under unsupervised or supervised or reinforcement learning?	Unsupervised learning finds hidden patterns or groupings in data with no reward or correct answer involved, like clustering customers by spending behavior in instagram or amazon. Reinforcement learning involves an agent taking actions in an environment and learning through reward or penalty feedback over time, like a robot learning to walk through trial and error. For stock prices, if you're predicting tomorrow's price using historical labeled data, that's supervised learning. If you're just grouping similar stocks by behavior with no prediction target, that's unsupervised. And if you're building a trading agent that learns buy/sell decisions based on profit or loss feedback over many trades, that's reinforcement learning.
Hi Muaz, To make linear separation, there might be some values in positive and negative side. As professor said we need to do correction in some values to reduce blue error and red error values. then output might be different.	Hi,by that professor meant the value of W is changed/ learned based on the errors
what is sign function	sign(Wx+b) = +1 if Wx+b = positive number sign(Wx+b) = -1 if Wx+b = begative number sign(Wx+b) = 0 if Wx+b = 0
Does algorithm ensure guaranteed convergence and also is there a closed form function to see what should be the data availibiity or data sufficiency for the convergence	Perceptron converges in finitely many updates if and only if the data is linearly separable. No theres no closed frm rule to deermin mdata is enoug
or incase of svm it is mandate to transform to 2 dimensional space - or it can visualize multidimensional separation boundaries	2D is just used in slides/lectures because it's easy to draw on a board. Mathematically, SVM works natively in any number of dimensions

what is the optimizer and loss function for svm	loss fuynction is hinge Loss, which penalizes predictions that are either wrong or not confidently correct it specifically wants data points to be on the right side of the margin with enough distance, not just barely correct. since SVM's objective is a convex quadratic optimization problem, not typically gradient descent in the classic sense. for large datasets variants like Stochastic Gradient Descent or specialized solvers like Sequential Minimal Optimization are used to make training faster and more scalable.
Muazul Hasan Khan_TA 10:37 AM No nothing is manual, the weights are kearnedby optimization algorithm such as gradient descent In the case, update is basically happening via learning mechanism and close to trial and error method ? is it ?	I mean trial and error wont be the right word its the algorithm which sequentially tunes the value of parameters to reduce the errors
Is pereptron equation applicable for all the type of supervised model? e.g. Linear regression, XG boost, Random Forest?	Perceptron's equation only applies to models that are literally structured as "linear combination of features passed through a decision function" (linear classifiers, logistic regression-ish family). Tree-based models (Random Forest, XGBoost) work completely differently; by splitting data based on feature thresholds, not by learning weights for a linear equation. So each model family has its own training algorithm suited to its own structure.
So in all these classification, our dataset should be well defined with all the attributes as feature. X will be a defined dataset & we know Y also what would be the output. This function keeps changing for different requirements. Identifying the function is trial & error and we some known function with which we can try with. If nothing suites , then we will customize a function for our needs. Hope this understanding is correct.	so functions you will get to know with practice. But there are generic rules as to when to use supervised or unsupervised, among supervised when to use linerar vs logistic etc. You will get that with practice
Why W & X has same dimension? Id W is m*n then X is n*p right?	Because W·X is a dot product which we multiply corresponding elements and sum them up. Dot product mathematically requires both vectors to have equal length; there's no "transforming dimensions" happening here, just a weighted sum.
Can you please include some real case studies in the session? Today is the third lecture, and it seems to me that everything is very much conceptual and theoretical. In the end of the lecture, another professor will come to tell the Python code. This approach does not make the course practical. I came to this course because it is "Applied" AI and ML course.	Hi Manik we understand your concern, can you talk to the program coordinator and explain the same post class?

What's the difference between using sign($W \cdot X_i$) versus using a threshold like $W \cdot X_i > 0$? in Perceptron Algorithm	both just check which side of the hyperplane a point is on. sign($W \cdot X_i$) directly outputs +1 or -1 (matching the label format of y_i). This is convenient because the perceptron's misclassification check is $y_i \cdot \text{sign}(W \cdot X_i) < 0$ $W \cdot X_i > 0$ gives you a True/False boolean; you'd still need an extra step to convert that into +1/-1 before you could compare it against y_i the same way.
in perceptron algorithm , the input vector can only have 1 feature and 1 bias or can it have more. And in this algorithm why is the weight updated with W + yiXi , how will that lead to convergence?	Yes,the algorithm works the same with any number of features. Also, it only works because the data is separable (margin $\gamma > 0$ exists). If not separable, there's no such W*, the bound breaks down, perceptron loops forever.
Question 1: when we are talking about Lasso or Rigid regularization, we look into the equation where we are adding the weights, does the weights have same dimensions? If no then how is this dimensionly correct. If yes, then could you please clarify my 2nd question Question 2: The weights can have same dimensions only in the situtation where the input variables are normalize, in case we normalize/scale them, then how do we decide which one to use, as you mentioned that we shouldn't use Rigid because a larger number multiplied with anything will result in some value which will have a contribution to the output, in scaling/normalizing all the input values result will be between certain values (like 0-1), hence how do we decide on this which one to use? Also other factor where we can decided Lasso is when we want to reduce the feature we can use Lasso, so is it something we have to take the decision on wether we want to reduce the feature or not?	you mean Ridge regression. ans to Q1 : yeah each weight is just a single scalar number tied to one feature, so when you sum $\|w\|$ or w^2 across all of them in the penalty term, you're just adding scalars together, not mismatched dimensions. So dimensionally it's fine regardless of feature scale, each w_i is one number per feature. Anbs to Q2 - the normalization point applies to both lasso and ridge equally, without scaling, bigger magnitude features get penalized unfairly more or less in both cases. So once you normalize, that specific issue goes away for either one, it's not really the deciding factor between lasso vs ridge.
in perceptron algorithm, the size of vector W is decided by size of vector X?	Yes, If your feature vector X has D features (plus the bias term, so D+1 total), then W also has D+1 elements; one weight per feature, plus one for the bias.
how did sign(-3) become -1 ?	sign(Wx+b) = +1 if Wx+b = positive number sign(Wx+b) = -1 if Wx+b = begative number sign(Wx+b) = 0 if Wx+b = 0
how does overlapping class - relates to real practical example - is it NLP type problem?	even in non NLPs the overlapping classes exist, e.g someone being both a price bargainer and a premium buyer when situation demands
If the data is not linearly separable, will the perceptron ever converge?	No, perceptron will never converge
why are we taking -1 , +1 , is it random ?	I mean what from what we know uptill now we could think of it as it gives the relative position of a data point corresponding to a decisio boundary

I guess we are understanding concepts in this module...but we will understand them more if we get to actually apply them...what is your experience/ how would u guide us when it comes to working on available dataset and applying these concepts? How much necessary is that?	yes look for ways to learn where you can apply them. Check out Kaggle, you will have datasets from all industries in there, you can use these concepts you learned here to apply it on the datasets from the industry you like and solve problems using those datasets
what is W in W=W=yX. Is it weight ? . If weight then it will be different for different features right ?	W is the weight vector whose each element corresponds to a different feature, and they do get different values.
or overlapping class primarily relates to image and nlp problem - not traditional business problem	overlaps happen everywhere
Cant logistic regression and classifier be used for same job? like predicting the classes?	yes we have multinomial logistic to predict more than 1 class you can use either then
how is the addition of yixi improving the W?	When a point is misclassified, it means W is "pointing the wrong way" relative to that point. Adding y·X to W literally nudges the weight vector to rotate toward that misclassified point's correct direction
is W is random selection and keep looking for the best fit?	Yes in practice W is randomly initialized and is adjusted to reduce error sequentially
in perceptron algorithm , the input vector can only have 1 feature and 1 bias or can it have more. And in this algorithm why is the weight updated with W + yiXi , how will that lead to convergence?	input vector can have as many features as you want, not just 1. so if you have D features, your weight vector W also has D dimensions (plus the bias, so D+1 like we discussed earlier). the 1 feature example is just the simplest case people use to explain it visually. on the update rule, W = W + yi*Xi is only applied when a point is misclassified. the logic is pretty intuitive if you think geometrically. the weight vector W is basically the normal vector to your decision boundary. if a point is misclassified, that means the dot product W·X landed on the wrong side (wrong sign compared to true label yi). adding yi*Xi to W literally nudges the weight vector in the direction that would make W·Xi more aligned with yi. like if yi is +1 but your current W gave a negative score, adding +Xi to W increases the dot product value for that point next time, pushing it toward being classified correctly. same logic works for negative label, you're nudging the boundary away from i
will perceptron cares about outliers?	Yes, perceptron is very sensitive to outliers
What if i have features of tiger in addition to cat and dog , how system will decide its neither cat or dog as everything will be -1 or +1	it will somehow try to forcefit to cat score in a 2 class etup./ to handle a 3rd class like tiger properly, you'd need a multi-class setup, like the one-vs-rest approach we talked about earlier, train separate cat-vs-rest, dog-vs-rest, tiger-vs-rest boundaries, then whichever one is most confident wins. plain single perceptron just isn't built to say "none of the above," it'll always force an output into whatever labels it was trained on.
So do we use the confusion matrix to count the errors after each iterations ?	during training, perceptron just tracks a simple error count (how many points are still misclassified in that iteration). confusion matrix is one of the evaluation tool used after training is done, over teat data

isnt the current example showing that both feature falling beyond above the hyper plane, this is the limitation of this algorithm isn't it, i dont understand the explanation provided currently	Yes you are correct perceptron fails when dataset is not linearly separable
Seems Perceptron algo is suited only for Liner classification ?	Yes only when dataset is linearly separable
one data set is linearly sepeatable or not - can I come to know by correlation matrix? if correlation is high then feature is liearly correlation with output(y). Can you let me know?	Correlation measures a linear association between one feature and y. Linear separability is different, it's about whether a hyperplane (combining ALL features together) can perfectly split the two classes. correlation gives a rough signal about individual feature relevance, but it's not a real test for linear separability
what do you mean by the last slide where it is a linear classifier is the best which is far from examples?	The separator farthest from the closest examples is best because it is most robust to noise or small shifts in the data and generalizes better to unseen points.
Sir, Could you please clarify how are we calculating the dot product of Sign(W.Xi+1)	W = [2, -1, 0.5] X_i = [3, 4, 1] ← note the last "1" is the bias term Dot product: $W \cdot X_i = (w1 \cdot x1) + (w2 \cdot x2) + (w3 \cdot 1)$ $= (2 \cdot 3) + (-1 \cdot 4) + (0.5 \cdot 1)$ $= 6 - 4 + 0.5$ $= 2.5$ Apply sign: sign(2.5) = +1 So the model predicts class +1 for this point. Without "1", you'd only have w1x1 + w2x2, a line forced through the origin. By treating bias as just another feature (always = 1) multiplied by its own learnable weight (w3), you let the boundary shift away from the origin, same trick as the intercept in y = mx + c.
what is the mathematical formulation for gradient descent optimizer	basic gradient descent update rule is: $W_{new} = W_{old} - \eta * \nabla L(W_{old})$ breaking that down: W is the weight/parameter you're updating η (eta) is the learning rate, controls how big a step you take $\nabla L(W)$ is the gradient of the loss function with respect to W, basically the slope telling you which direction increases loss

is it a parametric or non-parametric problem	Linear classifiers (perceptron, logistic, linear SVM) are parametric. Kernel SVM is often called non-parametric because the model depends on the support vectors, whose number grows with the data. kNN and decision trees are non-parametric.
does svm use batch or mini quadratic optimization	traditional SVM uses full-batch, it solves the whole quadratic programming (QP) problem using all training data at once, not mini-batches. that's actually why classic SVM doesn't scale well to huge datasets, we touched on this earlier, that $O(n^2)$ to $O(n^3)$ complexity comes from solving this big QP problem over the entire dataset in one shot.
If data is not linearly seprable what will happend to perceptron algo result?	if data isn't linearly separable, perceptron will never converge, it just keeps looping forever, updating weights every time it hits a misclassified point, but since no straight line can ever perfectly separate the classes, there will always be at least one point it keeps getting wrong no matter how it adjusts.
Can either logistic regression and Linear classifier be used for finding the classes? if yes, in what scenarios they differ in selecting?	Both are linear classifiers. Choose logistic regression when you need probabilities or interpretable odds. Use perceptron/SVM-style linear classifiers when you only need the class label (SVM adds margin robustness).
how do yi , +1 or -1 is predefined? Is it a labelled training dataset ?	Since its a supervised learning, y_i (+1 or -1) comes from your labeled training dataset. It's not something the algorithm figures out; it's given to you beforehand as the ground truth label for each data point.
can the professor explain in how multiple classes are handled	There are two ways one vs one and one vs all. That means vehicle - Car, Bike Bus, Three models (Car vs Bike, Bike vs Bus, Car vs Bus) and then majority voting, Second is One vs all Three mdoels here as well
I could not understand the initialization here. What initialization is being talked in Margin?	Once perceptron finds some W that perfectly separates the data, the question becomes: "but there are many possible separating lines; which one is best?" Margin = how far that line is from the nearest points of each class. The idea is: among all valid separators, prefer the one with the biggest margin (most "breathing room"), because it's more robust to noise. They're not the same "initialization". perceptron's initialization is about starting the search; margin is evaluated only on an already-found, fully-trained separating line.
can this concept be repeated again	please watch the recording, as given the time constraints we will not be able to repeat

in solution 2 the adustment will be more and a closer weight will be selected. So prably only few iterations could be larger. but how come we confidently say that the iterations will be surely higher in solution 2?	here's the thing, plain perceptron doesn't care about maximizing margin at all, it just stops updating the moment it finds ANY line that correctly separates all points. so it could land on something like solution 1 (small margin) just as easily as solution 2, whichever boundary its greedy update path happens to stumble into first, it has zero preference for bigger margin. the "solution 2 needs more iterations/adjustment" idea really comes from SVM, not perceptron, since SVM's objective specifically is to maximize margin, which is a harder constrained optimization (QP) problem, it's not just "stop at first success" like perceptron, it keeps adjusting until it finds the genuinely widest margin boundary. so you can't confidently say plain perceptron will take more iterations to reach solution 2, that claim only holds if you're talking about SVM's margin-maximizing process specifically.
Wouldn’t it be simple to say the bigger the margin, better the classifications measure for linearly separable datasets ?	I mean we cant say that exactly, classificationa accuracy depend on decision boundary itself, so there could by multiple hyperplanes (Decision boundaries which separate the data), but we choose the one with maximum margin that may generalize better then others
Is there a way the loss function/cost function or weight update function can be modified to coverge to certain. kind of solutions?	this is actually exactly what different algorithms are doing under the hood, same basic gradient descent machinery, but different loss functions steer convergence toward different kinds of solutions. e.g SVM does exactly this by using hinge loss plus a margin maximization term it specifically steers convergence toward the widest margin solution instead of just any separating line like perceptron does.
what is total margin 2k/|W|2??	distance from a point/plane to a parallel plane. By symmetry, the distance to the other side (blue boundary) is also k / ||W||. Total margin = distance to one side + distance to the other side = (k/||W||) + (k/||W||) = 2k / ||W||
can youtell me , what is the real life example for this ?	real life example for max margin linear classification couyld be email spam filtering, classifying some inbox messages as junk in facebook, linkedin etc

what is the mathematical formula for loss function optimization - is it batch data or single shot	Batch Gradient Descent (full batch): uses the WHOLE dataset every single update step. W_new = W_old - η * (1/N) * Σ ∇L(W_old, xi, yi) for all N samples . Stochastic Gradient Descent (SGD): uses just ONE random sample per update, single shot style. W_new = W_old - η * ∇L(W_old, xi, yi) for just one (xi, yi). Mini-batch Gradient Descent: middle ground, uses a small batch (like 32, 64, 128 samples) per update, this is actually what's used most in practice for deep learning. W_new = W_old - η * (1/m) * Σ ∇L(W_old, xi, yi) for m samples in that mini-batch you do not have to worry about the maths, TensorFlow/PyTorch will auto calculate
Is there a tool by which the equation of this line gets calculated based on the dataset/ labelled vector data and all iterations are done to get perfect or least error line. or we have to do it manually?	yes using sckit learn in python this is literally what SVM libraries do automatically. You just feed in your labeled data (X and y), and the solver internally runs the optimization to find the best W and k values that maximize margin while keeping classification correct, or as correct as possible with soft margin.
what is K here ? cant we take -1 and +1?	yes, you can set k =1
why WX = k or -k if it is still able to classify 100%? if it is able to classify 100% WX should be 0 right?	Decision boundary- W·X = 0; this is what's used for classification. W·X = k and W·X = −k are just reference markers showing how far the nearest points are from the real decision boundary (W·X=0). The closest "red" (+1) point sits exactly on W·X = k. The closest "blue" (−1) point sits exactly on W·X = −k.
if we do loss function minimization at batch level - how do we adjust the weight at each step	with batch gradient descent you don't update weights after every single data point you first compute the loss and gradient for ENTIRE batch, average it, then update weights just once per batch. step by step, for every sample in the batch, you calculate the prediction, compare to actual label to get individual error, then compute gradient of loss with respect to W for that sample. once you've done that for all samples in the batch, you average all those gradients together into one single combined gradient, then apply just ONE weight update using that averaged gradient.
or its optimization of the different node function in case of neural network and it works for single step	in a neural network, each "node" isn't really optimized separately on its own, it's all one connected computation. when you do a forward pass, data flows through all the layers/nodes to produce an output, you get one loss value at the end, not separate losses per node and thhen backpropagation calculates the gradient of that single loss with respect to EVERY weight in the network, layer by layer, going backward using chain rule, so every single weight in every node gets its own specific gradient value, but they all come from that same one overall loss, not independent per-node optimization.

What is the need for taking maximum margin? how does it impact?	Maximizing the margin means choosing the separator that is as far as possible from both classes. So, Bigger margin = more confident, more robust, better-generalizing classifier
Why K = 1? Does it make sense to use the closest euclidian distance of the points belonging to seperate classes to derive K?	it's simply a scaling convention
why we take w.x = 0 for the classifier line? doesnt it make all the weight values 0	W·X = 0 does not mean W itself is zero. It means: "find all the points X where this equation evaluates to zero."
What does the K represents?	it marks exactly how far out the closest points of each class sit from the decision boundary (in W·X units, not raw distance).
so , we can randomly choose K ? , if not then , why ?	k's magnitude is arbitrary, that's why we just default to 1. But it's not "randomly chosen" in the sense of varying or being zero/negative
how do we choose K ? any fomula to choose K	No formula needed for k, just fix it at 1 by convention.
Any formula to choose the right K ?	k is not computed, it's just fixed to k = 1 by convention, because of the scale-invariance between W and k (any positive k works, you'd just get a correspondingly rescaled W, same final boundary).
Why was w.x=k and -k was considered (two lines from optimal margin)? why was that used? please explain	We want to find the separator with the widest possible gap between the two classes (max margin). But to "maximize" something, you first need a way to measure it as a number/formula. The decision boundary alone (W·X=0) just tells you which side a point is on, doesn't tell you how much empty space exists around it. You can't compute a margin from a single line. so, the fix is to introduce two parallel reference lines: To measure the gap, we draw two more lines, parallel to the decision boundary, each touching the closest point of each class: W·X = k → touches the nearest +1 class point W·X = −k → touches the nearest −1 class point
What was the name of the algorithm to optimize Wmm again.. could not hear it	Quadratic Programming (QP) specifically solved in practice using SMO (Sequential Minimal Optimization)
What is the practical example/application of this margin optimization Formulation?	spam mail detection, medical diagnosis, credit card fraud detection etc.,
Got the point that we are maximizing the margin. But what are we trying to minimize? what why we took reciprocal?	Maximizing margin (2/||W||) and minimizing ||W|| are mathematically the same goal. we just reformulate it as a minimization of a clean, quadratic ½||W||² because that's a well-understood, solvable problem (quadratic programming), rather than directly maximizing an awkward fraction.
could you please tell me again why QP solvers are used?	Quadratic objective function with linear constraints, that's the exact textbook definition of a quadratic programming problem, so it naturally needs a QP solver, not regular gradient descent.

Hi, When you share the ppt in the resource section, can you remove all these writings done by professor. It is very difficult to find particular points using search to go into a particular concept, as it is just time consuming to find the topic that I need to learn.	Noted. We will upload both versions so that learners who would like to refer to the annotated PPT can access it as well.
Are these algorithms applicable only for classificaition problems or are these applicable for linear regression problems to find the better fit among the multiple possibilities	For plain OLS linear regression, there's actually no "multiple possibilities" problem the way classification problems have. The loss function as it is a sum of squared errors is strictly convex, meaning there's exactly ONE unique best fit line mathematically, no ambiguity to resolve like classification has with margin, so QP/margin concepts don't really apply here, you just solve it directly or via gradient descent and get one guaranteed answer.
so for classifier to measure margin max distance ...do we need to draw multiple parallel lines	No, for any single candidate decision boundary, you only ever need exactly 2 parallel lines ($W \cdot X=k$ and $W \cdot X=-k$), not multiple.
I did not understand constraint condition.....y(W.x) shd be greater than or equal to k	$y_i(W \cdot X_i) \geq k$ simply says: "Every point must not only be on the correct side, but far enough away (at least k) from the decision boundary" enforcing the margin for every single training point, for both classes, using one unified formula.
In the soft margin case, how do we decide on the acceptable number of the training observations misclassified or is there a separate concept to determine it ?	the soft margin formulation adds slack variables (ξi) for each point, allowing some points to violate the margin constraint, but each violation gets penalized in the objective function. the modified objective becomes something like minimizing $½\|\|W\|\|^2 + C * \Sigma\xi i$, so C controls how much you penalize those violations. high C: heavily penalizes misclassification, forces the model to classify almost everything correctly even if it means a much narrower margin, behaves closer to hard margin SVM. low C: tolerates more misclassification/slack, prioritizes a wider margin even if some points end up on the wrong side, more tolerant to noise/outliers.
Does regression work for discrete data or only continuos data?	Mostly continuous, but there are regression variants built specifically for discrete data: Poisson/negative binomial regression for count data (like number of complaints per day), and ordinal regression for ranked categories.
Is Max clissifier optimization is opposite of Gradient descent optimization calculation?	both are minimization at heart. The difference is just how the constraint is handled
Can you please provide comparison study of perceptron and logistic C	Logistic regression is smooth and probabilistic; with regularization (the default in sklearn) it converges reliably. The perceptron only converges on separable data and gives no probability.

Could explain the slack variable and C concept taking the spam example ?	slack variable is basically a "how much did this specific email violate the margin" measure. say legit boundary needs every spam email to score above some threshold k and every real email below -k. if one real email accidentally has a slightly spammy subject line and its score lands just barely on the wrong side, instead of forcing the whole model to drastically bend just to fix that one weird email, SVM assigns that email a small slack value , basically saying "yeah this one's a bit off, we're allowing it, but we're tracking how much." C parameter decides how harshly you punish that allowance overall. if you set high C, you're telling the model "I really don't want any misclassified emails, even weird borderline ones, fix them even if it means a narrow, tightly squeezed margin." this risks overfitting to noisy/weird emails, like one email with an unusual sender name ends up distorting the whole boundary just to get classified correctly.
If regression only works for continuous data and not discrete, then spam mail, medical diagnosis, and credit card fraud prediction represent the logistic regression where the input data is continuous but the outcome is discrete (Yes/No). Am I correct?	yes but they do not always use just logistic regression they use other methods also like NN
What are NN models?	neural network
what if a data point lies between optimal wx=0 line and wx=k line? what it is considered as? why is there a necessary to choose K value? what if we go ahead without that?	it's mathematically required to prevent the optimization from being degenerate. Without it, "minimize ||W||" has no finite, meaningful solution at all.
both are minimization at heart. The difference is just how the constraint is handled Could you please provide some elaborate explanation or some reference to understand it clearly?	at the core, gradient descent based methods (like linear regression, logistic regression, neural nets) are doing unconstrained minimization, you just have a loss function, and you keep moving in the direction that reduces it, no hard rules must be satisfied, if a point is slightly on the wrong side it's fine, the loss just reflects that error smoothly through the gradient. SVM's QP approach is doing constrained minimization, same core idea of minimizing something (½||W||²), but now there's a hard rule attached, yi(W.Xi) >= k must hold for every point, you can't just nudge weights freely, any update has to respect that boundary condition too.
so how do we tune C so that minimize these slack variables or penalise the obs in margin area? Any techniques that you could suggest?	C is a hyperparameter, so it's tuned empirically, not derived mathematically. GridSearchCV/RandomizedSearchCV

to maximize the margin, shd the nearest points on both sides be on equal distance from boundary? can't it be maximum if nearest pts on both sides are at different distances?	Think about it this way, say the nearest blue point is 3 units away and nearest red point is 7 units away from your current boundary line. Is this the maximum possible margin? No, you could just shift the entire boundary line 2 units closer to the red side, now blue point is 5 units away, red point is 5 units away, and your total margin width (distance between the two support points) stays exactly the same width-wise, but now you've used the available space more efficiently, actually you can then widen the margin lines themselves outward from that new centered position until they just touch the nearest points on both sides again, giving you more total margin than before. So if nearest points are unequal distances from the boundary, it means the boundary line isn't optimally placed yet, you're wasting potential margin space on one side. By definition, the TRUE max margin solution must have the boundary sitting exactly in the middle between both nearest points, since any shift away from the middle would mean y
how do you decide on grace marks?	For each point, after solving for the optimal W, the slack is simply: $\delta_i = \max(0, k - y_i(W \cdot X_i))$ If the point already satisfies the margin ($y_i(W \cdot X_i) \geq k$) → $\delta_i = 0$ (no grace mark needed, point is fine) If the point falls short (inside the margin or on the wrong side) → δ_i = exactly how much it's short by (k minus its actual score)
can u tell me please , why do we need delta ? if for outliers , then cant we remove them before hand ?	delta lets the model make that judgment call mathematically instead of you guessing manually beforehand. by adding δi as a learnable variable in the optimization itself, the model decides during training exactly how much "slack" each point needs, points that fit cleanly get δi = 0, points that are mildly difficult get a small δi, truly problematic ones get bigger δi, all of this gets solved simultaneously as part of the same optimization that's also finding the best W, no separate manual pre-filtering step needed.
So, when we caculate the outlier on the data set, internally how delta added in this context ?	δ (the "grace mark") isn't manually decided per point. it's automatically computed by the optimization itself, point by point, based on how far that point violates the margin.
We found K by maximizing the margin and call it K. Wouldn't outlier affect K? How will we ensure that the K is computed correctly?	yeah K itself doesn't get corrupted since it's fixed at 1 by convention, what actually gets affected by outliers is W. in hard margin SVM, even one outlier makes the problem unsolvable since it can't satisfy the strict constraint. soft margin fixes that by adding slack δi, so outliers get a penalty instead of breaking everything, and C controls how much W bends to accommodate them vs keeping a wider stable margin.
value of c how to choose , small and large is relative	C is a hyperparameter and we find the optimal value using cross validation
what is "argmin w" in the quation? how to interpret this?	argmin w basically means value of w which gives us the minimum vale of the expression

Also what i C , what is the need ? did not get it	C gives you that control, the regularization knob for SVM, and it's what you tune via cross-validation to get the best real-world performance.
For each point, after solving for the optimal W, the slack is simply: $\delta_i = \max(0, k - y_i(W \cdot X_i))$ If the point already satisfies the margin ($y_i(W \cdot X_i) \geq k$) → $\delta_i = 0$ (no grace mark needed, point is fine) If the point falls short (inside the margin or on the wrong side) → δ_i = exactly how much it's short by (k minus its actual score) in this case - what if it is large number , do you give grace to all ? or you change the function	This is known as hinge loss and is a part of the objective function. And the value of the parameters W is adjusted to minimize the objective function.
for each point calcuation, we can give seperate grace marks or same grace marks would be applied to all points?	standard SVM formulation uses one single C for everyone.
for each point calcuation, we can give seperate grace marks or same grace marks would be applied to all points?	Each point's grace mark (δ) is calculated separately.
Ok, but I am confused when we calculate the outlier by IQR = Q3-Q1, how that is related to here with grace mark ?	slack/grace mark outliers are a completely different idea, specific to classification, it's about a point that falls on the wrong side of the decision boundary or too close to it, violating the margin constraint, this has nothing to do with whether that point's raw feature values are statistically extreme or not. a point can be totally "normal" by IQR standards (nothing weird about its feature values) but still end up needing slack just because it happens to land near/across the boundary due to overlapping classes. so they're both called "outliers" in casual language, but IQR outlier = statistically unusual value in general data distribution, SVM slack outlier = misclassified/margin-violating point specifically relative to a decision boundary, these aren't the same mechanism or calculation at all, just similar sounding names for different problems.
what is the significance of alpha?	each αi is tied to one specific training point Xi, and its significance is basically "how much influence does this point have in defining the final boundary W." most points actually end up with αi = 0, meaning they contribute nothing to W at all, they're just regular points sitting comfortably away from the margin, totally ignored in the final formula.
But Prof. was saying different grace marks for each point	yes, each point gets its own, different grace mark (δ). C is the single shared "cost per unit of grace mark," the same for every point, while δ itself (the actual grace mark value) is indeed different for each point, exactly as your professor said. So, δ_i (grace mark) → different for every point, while C → one shared number, same for all points, controlling how much each unit of grace mark "costs" in the overall objective

I am asking about the classifier mechanism and how it relates to models.	so the classifier mechanism here is actually pretty universal across linear classifiers, you compute W.X (a weighted sum of features), then sign() just checks if that value is positive or negative to decide the class, +1 or -1. perceptron, logistic regression, and SVM all use this exact same core mechanism, compute a score, check which side of zero it lands on.
What is SVM?	support vector machine
what role SVM does it play in machine learning	SVM is a maximum-margin classifier: it picks the separating hyperplane farthest from both classes. With kernels it learns non-linear boundaries, and it also has a regression variant (SVR).
X1 is train data , and X test is test data ?	yes
what is the sign function? Do you mean sin function??	nope sign, look at the slide now
What are we testing with X1 and Xtest	X_i^TXtest measures similarity between the new point and each training point (support vector). These similarities, weighted by $\alpha_i y_i$, decide Xtest's class
Why does the classification rule depend only on support vectors and not all training points?	Only points that are "actively constraining" the margin get non-zero α; everyone else is irrelevant to the final boundary by the mathematical structure of the optimization itself.
Can you explain Classification ny SVM using some example?	already done on the email spam detection please refer to that
what it is kernel-based approach	Kernels are functions basically which project the data onto a high dimensional space
Y=Wi.Xi if SVM classifier W=0 then Y=X?	Nope . the classifier formula is Y = sign(W.X), not Y = W.X directly, and definitely not Y = X. if W = 0, then W.X = 0 for literally every single input X, no matter what X actually is, since anything multiplied by 0 is 0. sign(0) is typically defined as 0 (or undefined/arbitrary depending on convention), so you'd just get Y = 0 or some default class for every single input, not Y = X.
whats difference between SVM and linear classifiers?	it's more like linear classifiers is the broad family, and SVM is one specific member of that family, alongside perceptron and logistic regression which Sir has been discussing.
can you elaborate more on “Kernels are functions basically which project the data onto a high dimensional space” and link it to SVM?	Kernels represent an implicit projection to a higher-dimensional space where separation becomes easier. SVM is uniquely suited to exploit this because its entire solution and prediction rule are built entirely out of dot products, letting you substitute a kernel function in place of the dot product to get non-linear boundaries "for free," without ever explicitly computing the high-dimensional coordinates.
If two points are far apart in 2D but close in the transformed space, how does that affect classification?	ans rto Q1 : this is exactly the whole point of kernel tricks, two points that look far apart in the original space can become much closer (more similar) once transformed, if they share some hidden structure the kernel captures. Since SVM classifies based on distances/dot products in the transformed space, this new-found closeness is exactly what makes a previously non-linear boundary become linear.

so it is easy to say in complex prompts we give to models are always used as Non linear classifications	Both kernel SVM and neural networks (including LLMs) transform data into representations where complex, non-linear patterns become tractable. But SVM does this via an explicit kernel trick on a fixed input, while LLMs do this differeently
why is it called support vectors? any specific functionality?	as they support/anchor the position of the maximum-margin boundary.
if we are only calculating sign then , whay are we measuring simiarity between output	Similarity (via the kernel) is HOW the model decides; it's the substance of the computation, comparing the new point to known support vectors. Sign() just converts that accumulated evidence into a clean binary output.
Why are we multiplying the test X with all to compare it with them?	that's actually the whole mechanism of how SVM makes predictions, it needs to measure similarity between your new test point and every support vector (the important training points from before, remember most alphas are 0 so only support vectors actually matter). the dot product x^T.z literally measures how aligned two vectors are, if high similarity (large positive value), that support vector pulls the prediction toward its own class, if negative, it pushes away. the model combines all these similarity scores weighted by alpha to decide where the test point lands relative to the boundary.
how to identify which kernel is it?	start with linear, use visualization/domain intuition as a guide if available. or you can cross-validate across a few kernel options and let validation performance decide.
How does the SVM handle overlapping classes when even a non-linear kernel cannot perfectly separate them?	SVM uses the same soft-margin slack mechanism regardless of kernel, accepting a controlled tunable amount of misclassification (via C) protects against overfitting to inherently overlapping/noisy data.
Why is X2 not a support vector?	Support vectors are vectors which are at the margin or inside the margin
Why is the need for multiplying the train and test vectors? what is the significance?	Thats how we determine the position of a test point with respect to decision boundaries, dot product means similarity.
Here then what is the margin? The Margin was not defined in the question right?	margin doesn't disappear with kernels, it's just not explicitly written in that sign(W.X) prediction formula since that's just the "how to predict" step, not the "how we found W" step. margin is a gap between the boundary and the nearest points on each side, just now happening in the transformed kernel space instead of plain 2D.
Computationally this looks too exhaustive , does it mean all the feature vectors are used everytime for computation any new Xtest ?	Recall only support vectors have non-zero α_i, all other training points contribute zero and can be safely ignored.

what is difference between SVM and logistics regression as both is used for classification. Please also provide some real world example for differentiating	SVM preferred: Text/document classification (e.g., spam detection, sentiment classification with thousands of word-based features) or gene expression classification in bioinformatics (few hundred samples, thousands of features) as SVM handles high-dimensional, relatively small datasets well, and you mainly care about getting the classification right, not needing a precise probability. Logistic Regression preferred: Credit risk / loan default prediction at a bank as regulators and risk teams don't just want "approve/reject," they want an actual probability of default (e.g., "this applicant has a 23% chance of default") to make risk-adjusted decisions (like setting interest rates). They also need interpretable coefficients (e.g., "income increases odds of repayment by X%") to justify decisions to regulators; something SVM doesn't naturally provide. Short version: Pick SVM when you want a robust, margin-based hard classifier especially in high-dimensional data; pick Logistic Regression when you need calibrate
g(X) =(1/M) * (f1(X)+.....+fM(X)) what is M here?	number of models that is being considered , this is to take average
what is the need of ensemble average? in the end we will use one of the models only	A single model tends to overfit (high variance). Averaging many different models cancels out their individual errors/noise, giving a smoother, more reliable prediction than any single model alone
How bias will be negative? what does it mean in real world? if true value is “dog” but modal predict that image is close to cat as mean of error. then how we can think about negative bias?	bias being "negative" doesn't literally mean the number itself turns negative when dealing with classification labels like dog/cat, that language mostly makes sense for numeric/regression predictions. here what actually happens with classification bias is more about systematic directional errors, like if your model consistently misclassifies dog images as cat (maybe because training data had way more cats, or dog images were mostly blurry/dark), that's a bias, but it's describes as a pattern/tendency, not represented by a single negative number like in regression.
what is bias, can you give example? where that term applicable?	assuming a non linear relationship as a linear relationship is an eg of bias
what is Y an Y1, Y2 etc. in bias n variance?	Y1 is prediction from model 1, Y2 is prediction from model 2, and so on. Y is the one true correct answer
Is this the criteria we are expected to use (bias and variance) among the models for the choice of Kernes?	you mean kernels? the ans is not directly. bias-variance comes in indirectly through the behavior each kernel produces, not through some bias-variance number you compute to pick between kernels, like linear kernel tends toward higher bias (simpler, might underfit complex patterns) but lower variance (stable, less likely to overfit). RBF kernel can have lower bias (flexible, can fit complex boundaries) but higher variance risk (can overfit if gamma parameter set too aggressively).
by using undersampling will there any chance of data loss ?	Yes, undersampling causes real information loss, so it should be used carefully

How to perform over sampling and undersampling in python while we are working with dataset having class imbalance?	imbalanced-learn library
what is a closed form function in general?	A closed-form function/solution is one that can be computed directly using a finite, explicit mathematical expression
How will SVM help for >2 classes ?	using the voting techniques that was already explained. Reiterating : One-vs-Rest (OvR): train separate SVMs, dog-vs-rest, cat-vs-rest, rat-vs-rest, each one learns its own W and margin. for a new test point, run it through all three SVMs, each gives a score, whichever SVM is most confident (highest score) wins, that's the predicted class. One-vs-One (OvO): train SVM for every pair, dog-vs-cat, cat-vs-rat, dog-vs-rat, so for 3 classes that's 3 SVMs, for N classes it's N*(N-1)/2 SVMs. test point gets voted on by all pairs, whichever class wins the most pairwise matchups becomes the final prediction.'
In real time how we decide whether we should go for logistic regression or svm for this use case (telecom churn), or do we have try for both?	you can try both and see which gives better accuracy, most likely SVM would (not always though)
Which is a better approach one hot encoding or having a number assignment like 0,1,2,3..	The main factor is whether the category has a natural order. If it's ordinal (e.g., low/medium/high), numeric assignment (0,1,2) is fine since the order is meaningful. If it's nominal (e.g., city names, colors which has no inherent ranking), use one-hot encoding, because numeric assignment would falsely imply an order/ranking the model might learn from. Number of unique values matters too, but very high cardinality makes one-hot encoding expensive (too many columns), which might push you toward alternatives like target encoding.
If I keep gender_F column also then what would be the problem?	"dummy variable trap" as gender_F and gender_M carry perfectly redundant information (one is always 1 minus the other), which causes perfect multicollinearity. For models like linear/logistic regression this can make the underlying matrix singular (non-invertible), breaking the closed-form solution or wildly inflating coefficient variance. That's why we drop one dummy column to avoid that redundancy breaking the math.
what is random state , what else are the values that can be given in that?	it is to ensure that we can replicate the values again. By using the same value in this next time
How can we find IQR for multiple columns in a single go ?	using .quantile() on the whole DataFrame
why is the columns split needed here, gender wise we can consider 0 as F and 1 as M .....but im not getting the purpose of the split	we need numerical inputs to the model
What is the other real world examples where SVM can be used?	Medical Diagnosis Classifying whether a tumor is malignant or benign based on features from imaging/biopsy data (size, texture, shape measurements)
Where is the model created in the code looks like we are doing the prediction based on a model	The model is created at the line "svm_hard_rbf = SVC(kernel='rbf',...)", then trained with "svm_hard_rbf.fit(X_train_svm, y_train)", then custom function "evaluate_svm(...)" predict using the already-trained model.

why is SVM needed? when do we choose linear classifiers over SVM?	Choose a plain linear classifier when you need speed, interpretability, or a clear explanation of how X relates to Y. Choose SVM when you expect non-linear patterns or want the margin-maximization robustness, especially on high-dimensional data. SVM often performs comparably or better on complex/overlapping data, but always worth testing both and comparing validation performance.
On what basis Mauj is saying dataset is non seperable in code?	This dataset is not linearly separable. customer churn is a noisy, real-world business outcome (two customers can have nearly identical features), case of class overlap, exactly what we discussed with soft-margin SVM (C and slack variables)
With C the support vector will be higher? like the non zero alphas will be higher.. Means it will be more complex?	Higher C → narrower margin → fewer support vectors but a more tightly fitted, overfitting-prone boundary. Lower C → wider margin → more support vectors, smoother boundary.
When not to use SVM over other linear models like logistic ?	Choose logistic regression when you need speed, scale, interpretability, probabilities, or simple multi-class handling. Choose SVM when you specifically need robust margin-based separation on smaller/medium, high-dimensional datasets, and don't need probability outputs.
Didnt get how using radius we were able to draw a linear classifier	Okay, outer circle was of radius lets say 2, and the internal circle has radius 1, so now if we plot the points withrespect to radius on x-axis, we could get a linear decision boundary, try with pen and paper
Hi Muaz, In the last part, you have told that the linear separation is possible for the images shown where data distributed in spiral way. Can you please explain again?	Yeah sure, so we need to transform the data, we saw that they were concentric circles right, if we transform them to (r, theta) where r is the distance of a point from centre and theta is the angle, then the datasets becomes linearly separable
at last , you did Soft Margin SVM - Gaussian Kernel where F1 (Churn) 0.654 0.604 , so is that the best model result? or do we need to doe some more changes in C to achieve 0.893 0.872 , means near to 1?	We have an exercise later to tune the values of C and gamma but low performance on test set is due to dataset since it has overlapping classes
isnt hard margin same a liner regression in terms of outcome ?	No, they're fundamentally different in both purpose and outcome. Regression wants points close to the line (good fit), hard-margin SVM wants points far from the line (good separation/margin). They're solving completely different problems, even though both are "linear" techniques.
What if the circles are not concentric their centers are different? We just can take related to center of one circle right?	You could, but you'd have to manually figure out the right reference point, and it breaks down if the circles overlap or aren't cleanly nested around that point. This is a hand-crafted fix for one specific symmetric case, not a general solution. In practice, kernels like RBF avoid this problem entirely since they measure similarity directly between all pairs of data points, automatically adapt to circles/blobs/any other arrangement without you needing to know the shape beforehand.

can we plot the current churn data in 2d graph with hyperplane	Yes, but only by picking two features at a time (e.g., tenure and monthly charges) to plot on the x/y axes since you can't visualize all features simultaneously.